The Journal of Open Source Software

lptlib: A parallel Lagrangian particle tracking library for compressible CFD data and tracer-response analysis in optical velocimetry

Dilip Kalagotla [1,2¶] **and Paul D. Orkwis** [1]

1 Department of Aerospace Engineering and Engineering Mechanics, University of Cincinnati, Cincinnati, OH, USA **2** Florida Center for Advanced Aero-Propulsion, FAMU-FSU College of Engineering, Florida State University, Tallahassee, FL, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 29 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Laser-based velocity measurement never observes the fluid directly. Particle image velocimetry (PIV) seeds a flow with microscopic tracer particles, photographs them twice in quick succession, and infers the velocity field from how far they traveled in between (Raffel et al., 2018). The inference assumes that the particles go where the fluid goes, and in high-speed flow that assumption fails. A tracer crossing a shock wave carries too much inertia to decelerate as abruptly as the gas around it, so the measured velocity jump is smeared and the measured gradients are weaker than the real ones (Melling, 1997). The size of that bias depends on the particle, not on the instrument.

`lptlib` quantifies that bias. Given a computed flow field, the library releases particles of a specified size and density into it and integrates their trajectories, so the motion a camera would have recorded can be compared against the motion of the fluid itself. The difference between the two is the measurement error attributable to the tracer, shown in Figure 1 for three tracer sizes.

DRAFT

The Journal of Open Source Software

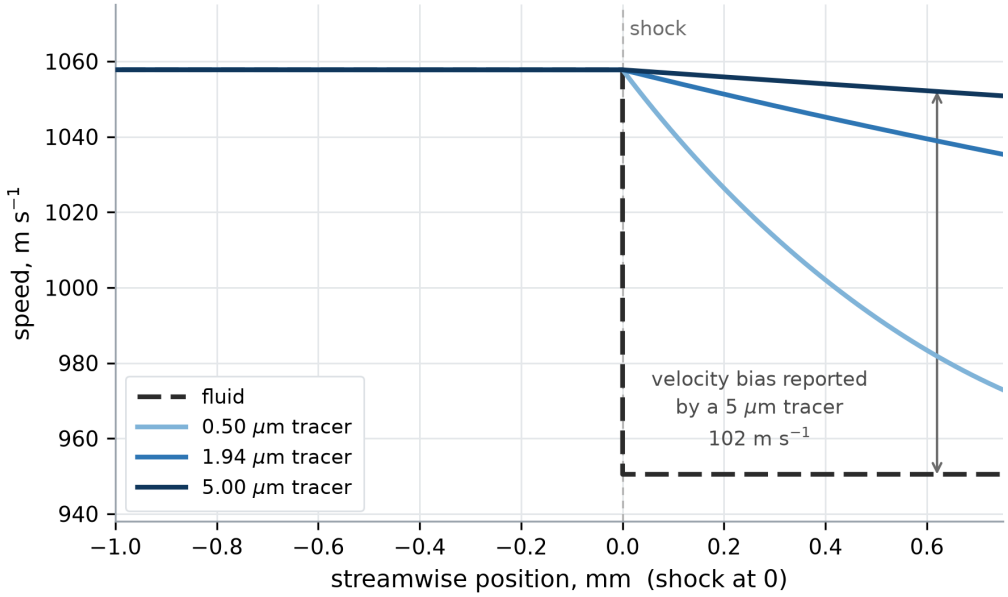


Figure 1: Inertial lag across an oblique shock, in the synthetic test case the library ships. The fluid decelerates abruptly at the shock, while tracers relax over a distance that grows with diameter, so a measurement taken just downstream reports a velocity that is too high. Mach 7.6, 20 degree deflection, Loth drag, tracer density 950 kg m^{-3}. Regenerated by `paper/make_figure.py`.

Working from a simulated flow rather than an experiment makes the comparison possible, because the true fluid velocity is known everywhere. The library reads structured, multi-block, curvilinear grids and flow solutions in the PLOT3D format (Walatka et al., 1990), interpolates the flow state to arbitrary points inside the curvilinear domain, and advances both fluid streamlines and inertial particle paths. Large ensembles are tracked in parallel through multiprocessing, thread pools, or MPI (Dalcin & Fang, 2021), and the scattered tracks are reduced back onto a user-defined Eulerian mesh to produce PIV-like fields. The tracker itself is not specific to velocimetry, and advances inertial particles through any resolved compressible field.

Statement of need

Correcting inertial bias requires a drag law that stays valid across every regime a tracer visits while crossing a shock. A 1 micron alumina particle in a Mach 5 flow passes from continuum to slip to near-free-molecular conditions within a few particle diameters, and no single classical closure covers that path. The practical questions the community asks are consequently comparative. How much does the answer change if the tracer is 300 nanometers instead of 1 micron, or if Henderson's closure (Henderson, 1976) is used rather than Loth's (Loth, 2008)? Answering either requires a tool where particle properties and the drag closure are both free parameters over the same flow field and the same integrator.

Such a tool has not been openly available. Researchers in this area have generally written their own single-purpose trackers, which are rarely released, rarely tested, and impossible to compare against one another. `lptlib` was built to close that gap. It exposes nine drag closures through a single argument, from Stokes and Oseen (Oseen, 1927) creeping flow through the Schiller-Naumann (Schiller & Naumann, 1933) and Melling (Melling, 1997) continuum corrections and a Cunningham-type slip correction (Cunningham, 1910) to the compressible and rarefied models of Henderson, Loth, Tedeschi (Tedeschi et al., 1999), and Subramaniam and Balachandar (Subramaniam & Balachandar, 2022), with temperature-dependent viscosity

DRAFT

from Keyes' correlation (Keyes, 1951) or Sutherland's law (Sutherland, 1893). All nine share one integrator, and the results reduce to a gridded field that downstream synthetic-imaging tools can consume. Optical velocimetry motivated the library and is what its shipped test cases cover, but nothing in the tracker is specific to it. The drag closures, the integrator, and the Eulerian reduction apply unchanged wherever inertial particles are advected through a resolved compressible field, from particle dispersion to erosion and impingement. The intended users are therefore experimentalists interpreting high-speed PIV data, and computational researchers studying one-way coupled particle-laden compressible flow.

State of the field

Open-source Lagrangian particle tracking is mature, but it is partitioned into communities whose assumptions do not transfer to compressible aerodynamics. Geophysical transport frameworks are the largest group. Parcels (Delandmeter & Sebille, 2019), OpenDrift (Dagestad et al., 2018), and MPTRAC (Hoffmann et al., 2025) advect particles through ocean or atmosphere velocity fields with great sophistication in interpolation and scale, but their particles are effectively passive, and the compressible and rarefied drag physics that dominates a tracer crossing a shock has no counterpart in them.

Experimental particle tracking velocimetry tools such as MyPTV (Shnapp, 2022) sit on the other side of the measurement. They consume camera images and produce trajectories, whereas `lptlib` consumes a CFD field and produces trajectories, then reduces them back to a grid. The two are complementary.

Closest in intent is ppiclF (Zwick, 2019), a parallel particle-in-cell library for particle-laden flow that scales to very large processor counts. However, ppiclF is written in Fortran and designed to be linked into a host solver, so it tracks particles during a simulation rather than after one. Solver-embedded Lagrangian cloud libraries such as those in OpenFOAM (Weller et al., 1998) share that constraint and a dependence on the solver's own mesh format. `lptlib` instead operates on a written solution file, so tracer response can be studied in an archived result without rerunning it, and the drag closure can be varied without touching the solver.

General-purpose visualization reaches part of this space. VTK and ParaView (Ahrens et al., 2005) read PLOT3D files and offer particle tracers, but those tracers integrate massless streamlines, so they answer where the fluid goes rather than where a tracer of a given size and density goes. Beyond visualization, open PLOT3D post-processing is dominated by legacy Fortran utilities and closed commercial tools.

Software design

Three decisions shape the library.

The first is adaptive integration keyed to trajectory curvature. Particle response near a shock is stiff, and a globally small time step is wasteful everywhere else, so `lptlib` adapts the step to the angle between successive displacements, refining where a path deflects sharply and coarsening again in smooth regions. Deflection was chosen over a conventional local error estimate because the quantity of interest is the tracer's spatial lag through a discontinuity, and deflection responds to the discontinuity directly rather than to the smoothness of the interpolated field. A blow-up detector guards the stiff branch, reduces the step, retries, and terminates the trajectory with a diagnostic if the step cannot be taken. The step is restored once integration succeeds again, so one shock crossing does not cripple the remainder of a path.

The second is one-way coupling. Particles do not act back on the fluid, which is accurate at the low seeding densities used in PIV and which makes every trajectory independent. Independence

DRAFT

allows the same tracking code to be driven by multiprocessing, a thread pool, or MPI without changing the numerics.

The third concerns input throughput. An ensemble reads the flow once and shares it across every particle, so the cost that matters is unsteady tracking, where a run loads a whole directory of PLOT3D solution files before a single trajectory is advanced. The PLOT3D reader loads each file in one buffered `numpy.fromfile` call and reconstructs multi-block arrays by strided slicing and Fortran-order reshaping, avoiding Python-level iteration over grid points (Harris et al., 2020; Virtanen et al., 2020). A reproducible benchmark under `benchmarks/` measures the effect, comparing readers only when they perform matched work. On a 2.04 million-point grid the strided reader beats an otherwise identical naive Python reader by a ratio of 27.8, with a 95 percent bootstrap interval of 26.1 to 28.8. Against a compiled Fortran reader doing the identical transpose it is about 1.4 times slower, while the full `read_grid`, which also builds the padded double-precision array and the per-block bounds, is about 1.75 times faster than Fortran doing that same work. Ratios rather than absolute times are reported, since absolute times move by more than an order of magnitude with page-cache state. `benchmarks/README.md` records the machine and explains how to add another.

Research impact statement

`lptlib` grew out of, and now underpins, a line of work on particle response and bias correction in supersonic PIV spanning 2020 to 2026. The methodology was first set out for turbomachinery CFD data in Kalagotla et al. (2020). The library then provided the tracking framework and PLOT3D post-processing interface of Kalagotla & Orkwis (2023) and Chhabra et al. (2023), produced the synthetic PIV data of Kalagotla & Orkwis (2024a), supplied the drag-model comparison and particle-size estimation of Kalagotla & Orkwis (2024b), and generated the training data behind the bias corrector of Kalagotla et al. (2025).

That thread has since appeared in peer-reviewed form. The neural bias corrector of Kalagotla, Cuppoletti, et al. (2026) was trained on 600 synthetic cases generated by this library and reduces inertial bias by up to 87 percent within its training distribution. The solver described there as in-house is `lptlib`, under the name it now carries.

Use now extends past the group that wrote the library. Two 2026 conference papers from Florida State University apply it to a supersonic wall-mounted hemisphere flow (Kalagotla, V. S., et al., 2026) and to uncertainty quantification of particle-inertia effects (Kalagotla, Oates, et al., 2026), and the co-authors on those papers took no part in writing it. Adoption by a wholly independent party is not claimed, since the library's first author is common to all of this work.

The library is installable from PyPI, documented, and covered by a test suite that runs on Linux and macOS across Python 3.10 to 3.13. Its verification tests check the implemented oblique-shock relations against an independently coded theta-beta-Mach solution and against published gas-dynamics tables. The particle integrator is checked against a high-accuracy reference integration of the same equation of motion, which it reproduces to a relative L2 velocity error of about 0.04 percent under both Stokes and standard sphere drag, and a physics-matched cross-code comparison against OpenFOAM parcel tracking is provided under `benchmarks/tracking/`.

AI usage disclosure

Generative AI coding assistants were used during the development of `lptlib`, specifically the Cursor editor's agent mode and Anthropic's Claude Code with the Claude Opus 4.6 and Claude Opus 5 models. They assisted with parts of the implementation, the test suite, the benchmarks, the documentation, and the preparation of this paper. All such output was reviewed, corrected

DRAFT

where necessary, and validated by the authors, who take full responsibility for the software and for the claims made here. The scientific formulation, the choice of models, the design decisions, and the interpretation of all results are the authors' own.

Acknowledgements

The authors thank Harpreet Chhabra for contributions to early test cases, and Daniel Cuppoletti for discussions on the experimental context. This work was carried out in the Department of Aerospace Engineering and Engineering Mechanics at the University of Cincinnati, which supported it financially. The University of Cincinnati had no involvement in the design of the software or in the preparation of this paper.

References

Ahrens, J., Geveci, B., & Law, C. (2005). ParaView: An end-user tool for large-data visualization. In *Visualization handbook* (pp. 717–731). Elsevier. https://doi.org/10.1016/B978-012387582-2/50038-1

Chhabra, H. S., Kalagotla, D., & Orkwis, P. D. (2023). Python based API to post-process CFD data. *AIAA SCITECH 2023 Forum*. https://doi.org/10.2514/6.2023-1225

Cunningham, E. (1910). On the velocity of steady fall of spherical particles through fluid medium. *Proceedings of the Royal Society of London. Series A, Containing Papers of a Mathematical and Physical Character*, *83*(563), 357–365. https://doi.org/10.1098/rspa.1910.0024

Dagestad, K.-F., Röhrs, J., Breivik, Ø., & Ådlandsvik, B. (2018). OpenDrift v1.0: A generic framework for trajectory modelling. *Geoscientific Model Development*, *11*(4), 1405–1420. https://doi.org/10.5194/gmd-11-1405-2018

Dalcin, L., & Fang, Y.-L. L. (2021). mpi4py: Status update after 12 years of development. *Computing in Science & Engineering*, *23*(4), 47–54. https://doi.org/10.1109/MCSE.2021.3083216

Delandmeter, P., & Sebille, E. van. (2019). The Parcels v2.0 Lagrangian framework: New field interpolation schemes. *Geoscientific Model Development*, *12*(8), 3571–3584. https://doi.org/10.5194/gmd-12-3571-2019

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Henderson, C. B. (1976). Drag coefficients of spheres in continuum and rarefied flows. *AIAA Journal*, *14*(6), 707–708. https://doi.org/10.2514/3.61409

Hoffmann, L., Clemens, J., Griessbach, S., Haghighi Mood, K., Heng, Y., Khosrawi, F., Liu, M., Lu, Y.-S., Meyer, C., Nobre Wittwer, N., Wu, X., & Zou, L. (2025). MPTRAC: A high-performance Lagrangian transport model for atmospheric air parcel dispersion. *Journal of Open Source Software*, *10*(111), 8177. https://doi.org/10.21105/joss.08177

Kalagotla, D., Cuppoletti, D. R., Orkwis, P. D., Hernandez-Lichtl, K., Gustavsson, J., & Kumar, R. (2026). Deep-learning-based particle inertia bias corrector for shock-dominated PIV data. *Experiments in Fluids*, *67*(5), 57. https://doi.org/10.1007/s00348-026-04216-z

Kalagotla, D., Oates, W., & Kumar, R. (2026). Uncertainty quantification of particle inertia effects in supersonic particle image velocimetry. *AIAA AVIATION 2026 Forum*. https:

DRAFT

//doi.org/10.2514/6.2026-4490

Kalagotla, D., & Orkwis, P. D. (2023). An object-oriented approach to tracking particles in a flow. *AIAA AVIATION 2023 Forum*. https://doi.org/10.2514/6.2023-3276

Kalagotla, D., & Orkwis, P. D. (2024a). A new approach to synthetic PIV data based on particle dynamics history. *AIAA SCITECH 2024 Forum*. https://doi.org/10.2514/6.2024-2667

Kalagotla, D., & Orkwis, P. D. (2024b). Numerical analysis of drag models and particle size estimation for PIV in supersonic flows. *AIAA AVIATION Forum and ASCEND 2024*. https://doi.org/10.2514/6.2024-4382

Kalagotla, D., Orkwis, P. D., & Cuppoletti, D. R. (2025). Deep learning based particle inertia bias corrector for particle image velocimetry. *AIAA SCITECH 2025 Forum*. https://doi.org/10.2514/6.2025-0476

Kalagotla, D., Orkwis, P. D., & Turner, M. (2020). Methodology to study the behavior of tracer particles in the flow field of rotor 37 using CFD data. *AIAA Scitech 2020 Forum*. https://doi.org/10.2514/6.2020-1809

Kalagotla, D., V. S., S., & Kumar, R. (2026). Influence of particle inertia on PIV measurements in supersonic wall-mounted hemisphere flow. *AIAA AVIATION 2026 Forum*. https://doi.org/10.2514/6.2026-4128

Keyes, F. G. (1951). A summary of viscosity and heat-conduction data for He, A, H2, O2, N2, CO, CO2, H2O, and air. *Transactions of the ASME*, *73*(5), 589–596.

Loth, E. (2008). Compressibility and rarefaction effects on drag of a spherical particle. *AIAA Journal*, *46*(9), 2219–2228. https://doi.org/10.2514/1.28943

Melling, A. (1997). Tracer particles and seeding for particle image velocimetry. *Measurement Science and Technology*, *8*(12), 1406–1416. https://doi.org/10.1088/0957-0233/8/12/005

Oseen, C. W. (1927). *Neuere methoden und ergebnisse in der hydrodynamik*. Akademische Verlagsgesellschaft.

Raffel, M., Willert, C. E., Scarano, F., Kähler, C. J., Wereley, S. T., & Kompenhans, J. (2018). *Particle image velocimetry: A practical guide* (3rd ed.). Springer International Publishing. https://doi.org/10.1007/978-3-319-68852-7

Schiller, L., & Naumann, A. (1933). Über die grundlegenden berechnungen bei der schwerkraftaufbereitung. *Zeitschrift Des Vereines Deutscher Ingenieure*, *77*, 318–320.

Shnapp, R. (2022). MyPTV: A Python package for 3D particle tracking. *Journal of Open Source Software*, *7*(75), 4398. https://doi.org/10.21105/joss.04398

Subramaniam, S., & Balachandar, S. (Eds.). (2022). *Modeling approaches and computational methods for particle-laden turbulent flows* (1st ed.). Elsevier. ISBN: 978-0-323-90133-8

Sutherland, W. (1893). LII. The viscosity of gases and molecular force. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, *36*(223), 507–531. https://doi.org/10.1080/14786449308620508

Tedeschi, G., Gouin, H., & Elena, M. (1999). Motion of tracer particles in supersonic flows. *Experiments in Fluids*, *26*(4), 288–296. https://doi.org/10.1007/s003480050291

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Walatka, P. P., Buning, P. G., Pierce, L., & Elson, P. A. (1990). *PLOT3D user's manual* (NASA-TM-101067). NASA Ames Research Center. https://ntrs.nasa.gov/citations/19900013774

DRAFT

Weller, H. G., Tabor, G., Jasak, H., & Fureby, C. (1998). A tensorial approach to computational continuum mechanics using object-oriented techniques. *Computers in Physics*, *12*(6), 620–631. https://doi.org/10.1063/1.168744

Zwick, D. (2019). ppiclF: A parallel particle-in-cell library in Fortran. *Journal of Open Source Software*, *4*(37), 1400. https://doi.org/10.21105/joss.01400

DRAFT